SENDA2 and three-dimensional loading: from classic routing to the integrated problem, measured honestly

Second educational guide to the LATTIMEX engine (version 1.1.0): what changed, how it was measured
and how to try it on your computer

Erick Martínez
LATTIMEX · independent research · Querétaro, Mexico
contacto@lattimex.com · https://lattimex.com

October 2026

Abstract

In a nutshell. The first guide opened SENDA, the LATTIMEX routing core, and measured it on the classic problem. This second guide reports the advance: a better core (SENDA2), a new three-dimensional packer (P3L) and their integration in engine 1.1.0, which is open source (MIT). Like the first one, it explains every piece from scratch and gives as much room to *how* things were measured as to the results, including our own mistakes that the measurement uncovered.

SENDA2 adds three changes to the adaptive large neighbourhood search of SENDA: a cached repair, discarding repairs that do not fit, and a 0.02 s cap on polishing each new best. In a single session, with the same clock and independent verification, it is +0.33 pp behind HGS-CVRP on 41 X instances at 4 s (1.74 % versus 1.40 %; FILO2, 0.72 %) and +0.07 pp on A/B/E at 2 s with a fixed fleet; the first guide reported a distance of about 2.6 pp on X. For loading, we built a harness against the published optima of Tamke et al. (2025) and the validator of Krebs and Ehmke (2023). It revealed that the 1.0.0 packer rejected 34 % to 61 % of routes that do fit and allowed a later box on top of an earlier one. P3L lowers that rejection to 24–36 % and, with the strict checker, the engine no longer returns packings that violate the unloading order. A routing search that checks the load at every step (R3L) achieves 285 of 285 valid solutions on the 19 Gendreau instances, at +6.9 % distance from Tamke's solutions (which ran up to 8 hours per instance). R3L does not scale to 300 deliveries; the document explains why and what remains open.

Keywords: vehicle routing; CVRP; 3L-CVRP; ALNS; three-dimensional packing; unloading order (LIFO); reproducible evaluation; optimization teaching.

Contents

How to read this document **3**

1 Introduction **3**

2 SENDA2: the routing core **4**
2.1 What changes with respect to SENDA 4
2.2 How we got there: two useful mistakes 5
2.3 Results against HGS-CVRP and FILO2 5
2.4 SENDA2 inside the engine 6

© 2026 Erick Martínez (LATTIMEX). CC BY 4.0: https://creativecommons.org/licenses/by/4.0/

3 Loading: measuring and building a packer **7**
3.1 The problem with three-dimensional loading . . . 7
3.2 How to measure a packer without fooling yourself . . . 8
3.3 What the measurement found in version 1.0.0 . . . 8
3.4 P3L . . . 8

4 Integration: routing that knows what fits **9**
4.1 Why a good packer is not enough . . . 9
4.2 R3L . . . 9
4.3 Results against Tamke . . . 10
4.4 At product scale . . . 11

5 Discussion **11**

6 Conclusions and future work **12**

7 Try it on your computer **12**

How to read this document

This guide continues the first one (*SENDA: vehicle routing with single-trajectory search, explained and honestly measured*, September 30, 2026, `paper/senda_guide_en.pdf`). It does not replace it: the first remains the reference to understand SENDA piece by piece (construction, granular local search, ALNS, segment moves, portfolio) and the basics of a fair comparison. Here that knowledge is assumed and only what is new is explained.

- Section 2 presents SENDA2, how we got there and how it compares with HGS-CVRP and FILO2 on the classic problem.
- Section 3 introduces the problem with three-dimensional loading, explains how to measure a packer without fooling yourself and presents P3L.
- Section 4 joins routing and packing: why a good packer is not enough, what R3L is and what engine 1.1.0 achieves.
- Section 7 explains how to use the full engine or each piece on its own.

The boxes have the same four roles as in the first guide:

Key idea

Summarizes the concept not to forget.

Worked example

Works out a small calculation, step by step, with concrete numbers.

Methodological lesson

Describes a possible mistake, or one that actually happened in this project, and how to avoid it.

To reproduce

Gives the SHA-256 fingerprints of the artifacts behind a figure and how to repeat the measurement.

About this edition. It accompanies version 1.1.0 of the public LATTIMEX repository. The repository contains the engine and this guide; it does not contain the comparison programs (HGS-CVRP, FILO2, Krebs's validator, the exact CP-SAT model) nor the campaign scripts. They are described here so that the comparison is transparent; raw data, contracts and logs are kept in the author's archive and each figure cites the fingerprint of the artifact behind it.

1 Introduction

The first guide ended with three open tasks: give the search more effective time on large instances, measure the core on the integrated problem with three-dimensional loading, and widen the benchmark. This guide reports what happened with the first two.

The short answer has three parts:

1. **SENDA2.** Three small changes to the ALNS of SENDA close most of the distance to HGS-CVRP on the X instances: from about 2.6 to 0.33 gap points at 4 s. The path included our own measurement error and a hypothesis (the *k-medoids* seed) that ablation ruled out.

2. **P3L and the strict checker.** Measuring the 1.0.0 packer against routes *known* to fit showed that it rejected half of them and that its unloading-order rule had a gap. P3L is a new packer with the standard semantics; the strict checker guarantees the engine never returns an invalid packing.

3. **Integration.** A good packer is not enough: if routing does not know what fits, it proposes impossible routes. R3L, a routing search that asks the packer at every step, solves validity on small instances; at product scale the engine uses P3L inside SENDA2 routing and warns when it cannot keep the unloading order.

Key idea

Every advance in this guide started with a measurement that contradicted what we believed. Version 1.0.0 was not worse than advertised at routing, but it was at loading: with unloading order it returned plans that violated it. Measuring against known optima with an independent judge is what made it visible.

2 SENDA2: the routing core

2.1 What changes with respect to SENDA

SENDA (first guide, Sections 3 to 7) follows a single trajectory per run: it constructs, improves with granular local search, restructures with an ALNS that destroys part of the solution and repairs it, polishes each new best and splits its time among two or three configurations. SENDA2 keeps all of that and changes three things inside the ALNS:

(a) **Cached regret repair.** When reinserting the removed customers, the repair of SENDA recomputes, for each customer and at every step, its best position in all routes. SENDA2 keeps a single candidate insertion per route for each customer (the cheapest among the positions next to its granular neighbours) and, after each insertion, only recomputes the route that changed. Each repair costs much less and the ALNS does more iterations in the same time.

(b) **Discarding failed repairs.** With a fixed fleet and no penalty, if the repair cannot fit every customer without exceeding capacity, SENDA2 discards the candidate (and penalizes the weights of the operators that produced it) instead of accepting it infeasible.

(c) **Polish cap.** Each time a new best appears, SENDA polishes it with local search until it no longer improves. SENDA2 limits that polishing to $\min(0.02\,\text{s},\ 0.20 \cdot t_{\text{rem}})$, where t_{rem} is the remaining time.

Worked example: how much time the polish cap frees

On X at 4 s, each solver has 80 % of the clock: 3.2 s. In the measurement before SENDA2, polishing each new best used on average about 1.9 of those 3.2 s, almost 60 %. With the 0.02 s cap, a trial that finds 40 new bests spends at most $40 \times 0.02 = 0.8$ s polishing, and the rest goes back to the ALNS. In the confirmation (Section 2.3), mean iterations on X went from 654 to 1 402 per trial and the gap dropped by 0.59 points.

2.2 How we got there: two useful mistakes

Methodological lesson: the degraded control

The first candidate, called *KM-R1*, seeded the initial solution with k-medoids and destroyed by medoids. Its first comparison against SENDA showed a large advantage. On repeating it we found that the control (SENDA) had run at half its normal iterations: another session, another machine load. Repeated with all three arms in the same session and with a *speed sentinel* (the control's iterations must match those of a reference run), the advantage was still real but smaller. Rule: **all arms in the same session, in rotated order, with a speed sentinel**; never against a number from another session.

Methodological lesson: the name that ablation removed

With the advantage confirmed, it was tempting to credit the k-medoids. Ablation (removing one piece at a time, in the same session) said otherwise: removing the seed or the medoid destruction changed nothing; removing the cached repair cost 0.87 points on X, and the polish cap explained most of the rest. The candidate was simplified to those pieces and lost the name "SENDA-KM": it is called SENDA2. Moreover, the 0.02 s cap was chosen by looking at one round's data, so its figure could not be published without repeating it with new seeds (Section 2.3).

2.3 Results against HGS-CVRP and FILO2

Design. As in the first guide: 60 A/B/E instances with a fixed fleet at 2 s and 41 X instances with a free fleet at 4 s; 80 % of the clock as internal budget for every solver; one thread; distances rounded to integers (CVRPLIB convention); independent verification of coverage, capacity, fleet and cost. HGS-CVRP [9] and FILO2 [1] were compiled from their official sources. The main table is a session with all three solvers and never-used seeds (3000–3900); the H1-C confirmation, with other new seeds (2000–2900), repeats the comparison with HGS.

Table 1: Mean gap to the best known solution (BKS). Same session, same clock. On A/B/E we average the 55 instances where all three solvers respected the fleet in every run (FILO2 has no fixed-fleet option and used one extra route on 5 instances).

	SENDA2	HGS-CVRP	FILO2
X, 41 instances, 4 s, 5 seeds	1.74 %	1.40 %	**0.72 %**
A/B/E, 55 instances, 2 s, 10 seeds	0.074 %	**0.006 %**	0.053 %

Paired comparisons. Per instance, Δ = gap of SENDA2 minus gap of the other solver (bootstrap 95 % CI; Wilcoxon test):

- **Against HGS, on X:** +0.33 pp [0.21, 0.45], $p = 3 \cdot 10^{-6}$; SENDA2 is better on 7 of 41 instances. The H1-C confirmation gives +0.34 pp with SENDA2 and +0.28 pp [0.13, 0.42] with a variant that kept the discarded pieces.
- **Against HGS, on A/B/E:** +0.068 pp [0.033, 0.111]; tie on 36 instances and HGS is better on 19.
- **Against FILO2:** on X, +1.02 pp [0.85, 1.17], and FILO2 wins on 40 of 41; on A/B/E no clear difference was detected (+0.021 pp, CI crossing zero).

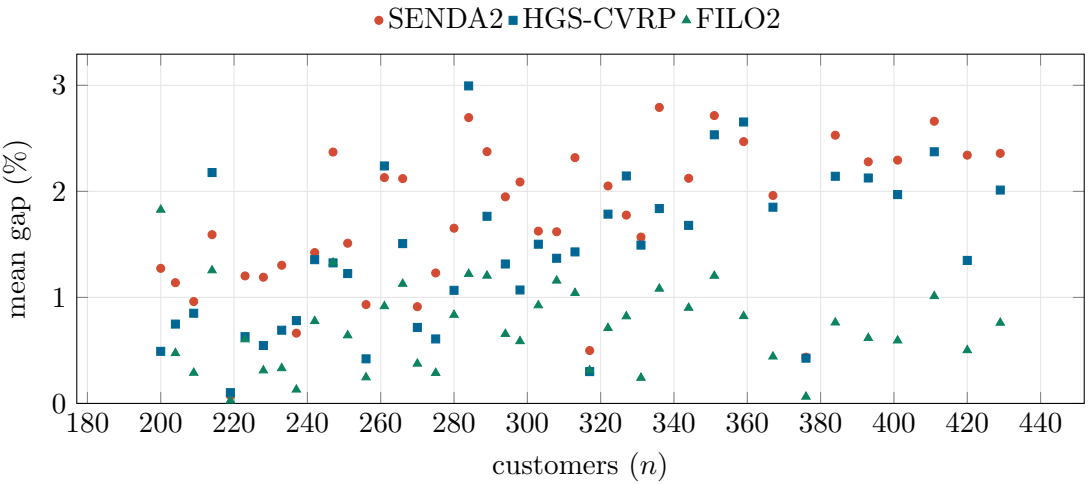


Figure 1: Mean gap per X instance at 4 s (5 seeds per solver, same session). FILO2's advantage grows with instance size.

Against SENDA. The first guide reported, on another machine, 4.119 % on X versus 1.450 % for HGS. To avoid mixing sessions, the honest comparison uses HGS as a common anchor: in an October session, SENDA was +2.64 pp behind HGS on X (4.05 % versus 1.41 %); SENDA2 is +0.33 pp behind. On A/B/E, from +0.094 to +0.068 pp.

Key idea

The honest claim is not "state of the art": FILO2 is clearly stronger on large instances with a short clock, and HGS is still better on average. SENDA2 is a core of about 2 200 lines, single trajectory, that respects a fixed fleet and stays close to HGS. Its value lies in being simple, auditable and carefully measured.

To reproduce

Session with all three solvers: `raw_trials.jsonl ef823740c317...`, contract `906e37b0ab29...`. H1-C confirmation: `2ac424ad3411...`. With the repository: `python -m lattimex cvrp INSTANCE.vrp -time 4` (X) or `-time 2 -fleet fixed` (A/B/E) splits the time and picks configurations as in the measurement. Repeated on the same benchmark with the published binary, one run at a time, it gave 1.77 % on X (41 instances, 1 seed) and 0.086 % on A/B/E (60 instances, 2 seeds), within seed-to-seed variation. **Run one instance at a time**: with six processes in parallel on the same machine the X gap rose to 1.96 %, because each process did fewer iterations.

2.4 SENDA2 inside the engine

The engine does not call the core the way the benchmark does: it splits the budget among routing, packing and re-optimization, uses an exact fleet and runs a portfolio of configurations on two threads. Measured that way (107 instances, 3 seeds, same session), SENDA2 versus SENDA:

- A/B/E: −0.036 % distance [−0.068, −0.009], $p = 0.02$.
- X: the result depends on how many customers each route serves (Spearman $\rho = -0.65$). With 12 or more customers per route, −0.89 % (wins on 14 of 16); with fewer than 6, +2.99 % (loses on 9 of 12).

Real LATTIMEX routes are long (30 to 75 deliveries per vehicle), so engine 1.1.0 uses SENDA2 by default and falls back to the 1.0.0 core when there are fewer than 6 deliveries per vehicle. The

rule was set with the same data that motivate it; it is an engineering decision, not a confirmed figure.

The same round found a risk in the exact-fleet constructor: when the load fills between 92.5 % and 99.7 % of total capacity, it could not split it into k vehicles and the engine rejected the order (11 of 41 X instances). Engine 1.1.0 adds a fallback that assigns accepting overload and removes it with local search; it solved all 11 in under half a second.

3 Loading: measuring and building a packer

3.1 The problem with three-dimensional loading

In the 3L-CVRP [3] each customer orders boxes with length, width, height, weight and a fragility flag, and each route must fit inside the vehicle's cargo space. We use Gendreau's axes: x is the length (the door is at $x = L$), y the width and z the height; boxes only rotate horizontally. Besides staying inside and not overlapping, a packing may require:

- **Support:** at least 75 % of each box's base rests on the floor or on boxes whose top face is at that height.
- **Fragility:** a non-fragile box cannot rest on a fragile one.
- **Unloading order (LIFO):** on reaching each customer, its boxes come out without moving those of later customers.

The five standard variants combine these rules:

Table 2: 3L-CVRP variants (Gendreau et al., 2006).

Variant	Support	Fragility	Unloading order
loading-only	–	–	–
no-support	–	yes	yes
no-fragility	yes	–	yes
no-lifo	yes	yes	–
all-constraints	yes	yes	yes

The unloading order, precisely. Let e be a box of a customer visited *before* the customer of box t. The packing violates the order if t blocks e in one of two ways (Figure 2): (i) t is *in front of* e, closer to the door, and their projections on the y–z plane touch; or (ii) t is *on top of* e and their projections on the x–y plane touch. These are the semantics of the public validator of Krebs and Ehmke [6, 5].

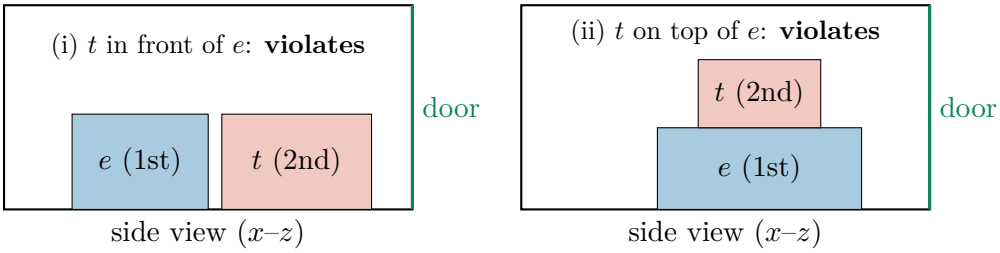


Figure 2: The two ways a box t of a later delivery blocks a box e of an earlier delivery. The 1.0.0 packer only forbade the first.

3.2 How to measure a packer without fooling yourself

Measuring a packer looks easy: give it routes and count how many it packs. The trap is that you do not know how many *should* fit. Given loose routes, any packer looks good; given impossible routes, any packer looks bad.

Routes known to fit. Tamke et al. [8] solved with an exact branch-and-cut the 19 classic Gendreau instances with up to 50 customers, in all five variants, and published their solutions with the packing of each route [7]. Each of those 648 routes fits: its packing exists and the validator approves it. If a packer rejects one of them, it is a **false rejection**, with no ambiguity. We call this test *level 1*.

An independent judge. Every accepted packing is checked with Krebs's validator, which is not part of the engine. Before using it as a judge we checked that it judges correctly: it approves Tamke's 95 solutions (19 instances × 5 variants), rejects the 76 solutions of loose variants evaluated with all constraints, and rejects the 19 cases with a forced overlap.

Methodological lesson: the 2.4 % that did not measure what it seemed

In July we had reported that the engine's packer rejected only 2.4 % of routes. That measurement used routes from another heuristic built with all constraints and tested them on the loosest variant: they were loose routes. With the optimal routes, which are tight, rejection was 34 to 64 %. A quality figure only means something together with the benchmark it was measured on.

3.3 What the measurement found in version 1.0.0

- **High false rejection:** between 36 % (loading-only) and 64 % (all-constraints) of the optimal routes, growing with route size: with 9 to 12 boxes, 47 of 49 rejected. With 1 024 restarts instead of the usual ones, all-constraints only dropped from 64 % to 60 %: the limit was the placement algorithm, not the effort.
- **A gap in the unloading order:** the packer forbade case (i) of Figure 2 but not (ii). Krebs rejected 27 packings the engine considered valid.
- **In the full engine,** with all constraints, 1 of 57 runs was valid: when a route did not fit, the engine silently relaxed the unloading order.

3.4 P3L

P3L (`native/p3l.cpp`) packs one route in one container with the semantics of Figure 2:

1. **Loading sequence.** With unloading order, boxes are loaded from the last customer to the first; within each customer, by volume, base area, height or longest side.
2. **Gravity placement.** For each box, candidate positions in x and y are generated (the origin, the end of each placed box and the opposite wall) and each one drops until it rests; the first one by a lexicographic rule wins (for example, smallest x, then smallest z).
3. **Beam search** over the sequence: the best partial sequences are kept according to their contact with walls and boxes.
4. **Iterated local search** over the loading sequence and rotations, from several starts, until everything is placed or the cap runs out (50 ms per route in the engine).

Methodological lesson: positions that seem enough and are not

To speed up, it is common to restrict positions to *normal patterns*: the sums of box lengths. It is correct without support or unloading order, but not with them. An exact CP-SAT model, used as a reference during development, "proved" impossible 5 routes that Tamke had packed. The cause was the normal patterns; with all integer positions in x and y when there is support, as Tamke does, the 5 routes fit.

Results at level 1. The 13 instances not used to tune P3L, with the frozen candidate (Table 3). The "P3L + CP-SAT" column adds the exact model for the routes P3L does not pack; it lowers rejection almost to zero, but it is not part of the engine: it depends on OR-Tools and does not finish on routes of 55 or more boxes.

Table 3: False rejection on Tamke's optimal routes, 13 confirmation instances. No packing accepted by P3L was rejected by the validator.

Variant	1.0.0 packer	**P3L** (engine 1.1.0)	P3L + CP-SAT
loading-only	34.1 %	23.9 %	1.1 %
no-support	42.2 %	33.3 %	0.0 %
no-fragility	57.6 %	32.6 %	0.0 %
no-lifo	41.1 %	35.6 %	6.7 %
all-constraints	61.3 %	30.1 %	0.0 %

At product scale. On 159 routes of 20 to 97 boxes produced by the engine on 12 Querétaro instances, with all constraints, the 1.0.0 packer rejected at least 70.8 % of the routes that fit and gave 9 invalid packings; P3L with 50 ms, at least 14.6 %, and with 500 ms, at least 3.8 %, with no invalid ones ("at least" because there are no optima here: a route counts as feasible if some method packed it). Without unloading order, both pack all of them.

The strict checker. Engine 1.1.0 checks every packing with the semantics of Figure 2 before accepting it, whether it comes from P3L or from the 1.0.0 packer, which is kept as a fallback because it can turn boxes vertically when allowed. Against Krebs's validator, the checker agreed on 2 436 of 2 436 packings, including 105 that Krebs rejects.

4 Integration: routing that knows what fits

4.1 Why a good packer is not enough

The engine routes with an effective volume capacity, a fraction of the vehicle's volume, and packs afterwards. If a route does not fit, a *mediation* tries a few changes: reverse it, rotate it or move a customer to another route. Replacing only the packer with P3L raised valid runs with all constraints from 1 to 20 of 57, but the gap of the valid ones did not change (+15 to +29 %) and the exact model proved impossible 1 665 routes that routing had proposed. Routing kept proposing routes without knowing whether they fit.

4.2 R3L

R3L (`lattimex/engine/r3l.py`) is a ruin-and-recreate routing search in the style of SISR [2] with simulated annealing [4], in which every insertion asks the packer:

Algorithm 1 R3L (outline)

Input: customers, distance matrix, container, variant, maximum fleet k, time T

$S \leftarrow$ rebuild from scratch; if there are SENDA2 routes, also from them, and keep the best
while time remains **do**
    $S' \leftarrow$ remove from S strings of nearby customers (SISR)
    for each removed customer, in random order or by volume or distance **do**
        try its positions from cheapest to most expensive;
        keep the first whose route **fits** (P3L); open a new route only if there are fewer than k
    end for
    accept S' with the simulated annealing criterion; keep the best complete solution
end while

Three details make it fast. (1) Checks are staged: one greedy P3L pass for every candidate position (0.3 ms on small routes) and beam search only for the cheapest position. (2) A cache stores every verdict; without unloading order the key is the set of customers, because the order does not change the packing. (3) With unloading order, if a route does not fit it is tried reversed: same distance, different unloading order.

Worked example: the reversed route

A route visits A and then B. A orders a non-fragile box that covers the whole floor; B, a fragile one of the same size. In the order A, B, B's box (second delivery) must be loaded first, at the back or below; since it covers the whole floor, it goes below and A's on top: a non-fragile box on a fragile one, forbidden. In the order B, A, A's box goes below and B's fragile one on top: allowed, and B comes out first without moving anything. The distance is the same; R3L only has to try the route both ways.

4.3 Results against Tamke

The 19 Gendreau instances, all five variants and three seeds (285 runs per arm), with an "up to k" fleet (k = available vehicles, as in Tamke). A run is *valid* if Krebs's validator approves routing and loading in the variant. The gap is the distance against Tamke's solution.

Table 4: Full engine against Tamke. "Easy": instances Tamke solved to optimality in under 1 000 s; on the rest, Tamke ran up to 8 hours without closing its own bound.

	valid	mean gap	easy gap	rest gap
Engine 1.0.0 (10 s, exact fleet)	81/285	+26.4 %	–	–
SENDA2 + R3L, research (10 s)	283/285	+8.7 %	+4.2 %	+13.5 %
SENDA2 + R3L, research (30 s)	285/285	+6.9 %	+3.7 %	+10.2 %
Engine 1.1.0, exact fleet (10 s)	145/285	+26.4 %	–	–
Engine 1.1.0, `fleet_mode=up_to` (10 s)	284/285	+8.1 %	+4.1 %	+12.3 %

With an exact fleet (the default), engine 1.1.0 does not use R3L: it routes with SENDA2 and packs with P3L. It almost doubles the valid runs of engine 1.0.0, but the gap stays high, because it forces using every available vehicle while Tamke uses only those needed. When it relaxed the unloading order, the engine reported it in the response; the validator only rejected runs of that kind.

Methodological lesson: a goal that was not realistic

The initial goal was 100 % valid runs and gap $\leq$ 3 % in 10 s. Tamke's run times show it was not reasonable: on the hard instances, their exact method ran 8 hours without proving optimality. After seeing the first results the goal was adjusted to $\leq$ 3 % on the easy instances, and it is reported here together with the original. On the easy ones the mean is +3.7 % at 30 s (+3.3 % on the 13 instances not used to tune R3L): it is not met yet.

4.4 At product scale

R3L does not scale: each check repacks the whole route, and with routes of 60 to 75 boxes one check costs 15 to 300 ms instead of the 0.3 ms of Gendreau routes. With 300 deliveries, R3L did not even finish the initial solution in the available time. That is why engine 1.1.0 only uses it with an "up to k" fleet on small instances (up to 60 deliveries and 30 boxes per vehicle).

For large instances, engine 1.1.0 routes with SENDA2, packs with P3L in the requested variant, mediates with the strict unloading order and, only if a route still does not fit, packs it without unloading order **and reports it** in the response (`lifo_estricto`, `advertencias`). On the 12 product instances with all constraints (Table 5), version 1.0.0 declared every run feasible and none respected the unloading order.

Table 5: 12 product instances (60 to 300 deliveries, 1 to 10 vehicles), exact fleet, 30 s, 2 seeds. "Strict order": the engine declares it feasible, relaxed no route, and the validator approves the loading.

	Engine 1.0.0	**Engine 1.1.0**
all-constraints: strict order	0/24	19/24
all-constraints: relaxed and *reported*	–	5/24
loading-only: valid	24/24	24/24

Methodological lesson: "feasible" is not "correct"

Version 1.0.0 said "feasible" for plans that violated the unloading order, because it relaxed it without warning to avoid splitting routes. It was a reasonable design choice, but invisible to whoever read the response. Version 1.1.0 keeps relaxing as a last resort and makes it visible: a plan that forces the operator to move boxes should not look the same as one that does not.

5 Discussion

Where the value is. On the classic problem, SENDA2 brings LATTIMEX close to HGS without making it more complex. But inside the engine, with three-dimensional loading, the difference between routing cores is lost (round 2 detected no distance difference with 3D packing). The big gain for an operator is in loading: that the plan respects the unloading order or says when it cannot.

Limitations. (i) Levels 1 and 2 use the 19 Gendreau instances with up to 50 customers; at product scale there are no optima and the figures are bounds. (ii) P3L only rotates boxes horizontally; the 1.0.0 packer is kept as a fallback for boxes that may turn vertically, without a measurement of its own for that case. (iii) The rule choosing between SENDA and SENDA2 was set with the same data that motivate it. (iv) R3L was tuned on 6 instances and is also reported on the other 13, but the figures of Table 4 mix both groups. (v) All measurements are from a single machine.

6 Conclusions and future work

SENDA2 closes most of the distance to HGS on the classic CVRP with three small, measurable changes. P3L and the strict checker fix a real defect of version 1.0.0 and halve the rejection of routes that fit. R3L shows that checking the load inside routing solves validity on small instances.

Future work. (1) An *incremental* packing: when inserting a customer as the first delivery, its boxes are the last to be loaded and could be added without repacking the route; it is the missing piece for R3L to scale. (2) A stronger fast packer on tight routes, to meet the $\leq 3\,\%$ goal on the easy instances. (3) Localizing the local search of SENDA2 to the touched vertices, as FILO2 does, the most likely design difference behind the gap on X.

7 Try it on your computer

Engine 1.1.0 is published under the MIT license at `https://github.com/Erick-MF28/lattimex`; version 1.0.0 is still available at its tag. It can be used as a whole or in pieces:

```
git clone https://github.com/Erick-MF28/lattimex.git
cd lattimex
pip install -r requirements.txt  # Windows without compiler: pip install ziglang
python -m lattimex build         # compiles the cores and the packers
python -m lattimex probe         # deterministic fingerprint
python -m unittest discover -s tests/engine
```

1. SENDA2 alone, on CVRPLIB.

```
python -m lattimex cvrp X-n101-k25.vrp --time 4 --bks 27591
python -m lattimex cvrp A-n32-k5.vrp --time 2 --fleet fixed --sol A-n32-k5.sol
```

`-router senda` uses the 1.0.0 core, to compare both on your machine.

2. The packer alone. A JSON file describes the container and the boxes of a route, with `stop` = delivery position (1 = first):

```
{"cargo_cm": [300, 160, 160],
 "constraints": {"support": true, "fragility": true, "lifo": true},
 "items": [{"id": "A1", "dims_cm": [60, 40, 30], "stop": 1, "fragile": false},
           {"id": "B1", "dims_cm": [40, 30, 30], "stop": 2, "fragile": true}]}
```

`python -m lattimex pack boxes.json` returns the position of each box and whether the packing meets the variant.

3. The full engine. `python -m lattimex serve` and the LATTIMEX Planner at `https://lattimex.com/planner`, as in version 1.0.0: the Planner uses engine 1.1.0 without any change. Through the API, the field `fleet_mode` with value `up_to` allows using fewer vehicles (and enables R3L on small instances), and the field `router` with value `senda` forces the 1.0.0 core.

4. Repeating the loading measurement. Tamke's instances and solutions [7] (MIT) and Krebs's validator [5] (GPL-3.0) are obtained from their repositories; they are not part of LATTIMEX. The validator reads constraints by line number: generate the five variant files from its template. Tamke's solutions repeat the tour identifier, which must be renumbered before validating them.

To reproduce: artifacts

SENDA2, HGS and FILO2	`raw_trials.jsonl` `ef823740c317...`; contract `906e37b0ab29...`
H1-C confirmation	`raw_trials.jsonl` `2ac424ad3411...`
SENDA2 inside the engine (round 2)	`summary.json` `8dfb1d575211...`
Level 1 (1.0.0 packer and P3L)	`level1.jsonl` `1944dc5be91d...`; `level1_hyb_confirm.jsonl` `e2bde2f206bd...`
Integration (research)	`level2_s2r3l_summary.json` `ab706daaaabf...`
Published engine 1.1.0	`resumen.json` `12a78d4a782d...`
Figures in this document	data in `paper/datos2/` of this repository
Full artifacts	author's archive; shared on request (`contacto@lattimex.com`)

References

[1] Luca Accorsi and Daniele Vigo. Routing one million customers in a handful of minutes. arXiv:2306.14205, 2023.

[2] Jan Christiaens and Greet Vanden Berghe. Slack induction by string removals for vehicle routing problems. *Transportation Science*, 54(2):417–433, 2020.

[3] Michel Gendreau, Manuel Iori, Gilbert Laporte, and Silvano Martello. A tabu search algorithm for a routing and container loading problem. *Transportation Science*, 40(3):342–350, 2006.

[4] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[5] Corinna Krebs. SolutionValidator. https://github.com/CorinnaKrebs/SolutionValidator. GPL-3.0, commit 6a3b23f.

[6] Corinna Krebs and Jan Fabian Ehmke. Solution validator and visualizer for (combined) vehicle routing and container loading problems. *Annals of Operations Research*, 2023.

[7] Felix Tamke. 3L-CVRP: instances, solutions and code. https://github.com/felicze/3l-cvrp. MIT, commit 2b52d20.

[8] Felix Tamke, F. Linß, L. Kuttner, and Udo Buscher. A branch-and-cut algorithm for vehicle routing problems with three-dimensional loading constraints. *European Journal of Operational Research*, 2025.

[9] Thibaut Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140:105643, 2022.